Pi Coding Agent: A Practical Primer

Mick Cooney

16 October 2026

Table of Contents

1 The Short Version **2**
1.1 What This Primer Is Not . . . 3

2 How Pi Works **3**
2.1 Pi Is a Harness, Not a Model . . . 4
2.2 What Pi Provides . . . 4
2.3 Why Pi Is Built This Way . . . 5
2.4 Coming From Claude Code . . . 5

3 Installation and First Run **7**
3.1 A Controlled First Session . . . 8
3.2 Working in the Terminal . . . 9

4 Models and Authentication **10**
4.1 Hosted Authentication . . . 10
4.2 Hosted and Local Models . . . 11
4.2.1 OpenAI-Compatible Local Servers . . . 11
4.2.2 llama.cpp . . . 12

5 Running Pi Non-Interactively **12**

6 Customisation: Use the Smallest Layer That Works **15**
6.1 Context Files and Prompt Templates . . . 15
6.2 Skills . . . 16
6.3 Extensions . . . 16
6.4 Packages . . . 18
6.5 MCP and Subagents . . . 18
6.6 The Starter Suite: Install by Need . . . 19

7 Web Research: Assemble the Missing Capability **20**
7.1 Plain HTTP and Git Are Good for Static Sources . . . 20
7.2 Start With a Maintained Search Skill . . . 21
7.3 Add a Browser Only When Needed . . . 22

7.4 Search GitHub and Forums Directly . . . 23

8 Sessions, Hand-Off and Workflow 23
8.1 Sessions and Compaction . . . 24
8.2 Sharing and Hand-Off . . . 25
8.3 A Practical Workflow . . . 25

9 Safety and Containment 26
9.1 Project Trust . . . 26
9.2 Isolation Options . . . 27
9.3 Prompt Injection, Telemetry, and Reporting . . . 28

10 Pi Compared With Other Coding Agents 28
10.1 Pi Versus OpenCode and Aider . . . 29
10.2 Pi Versus Claude Code and Codex . . . 30
10.3 Forks and Derivatives . . . 30

11 Worked Example: A Reproducible Data-Science Project With Pi and Rocker 30
11.1 Project Goal and Contract . . . 31
11.2 Give Pi Bounded Tasks . . . 32
11.3 Build a Digest-Pinned Rocker Image . . . 33
11.4 Run It With Rootless Podman . . . 34
11.5 Verify the Result Independently . . . 35

12 Reference 36
12.1 Troubleshooting Order . . . 36
12.2 Configuration Map . . . 36
12.3 Glossary . . . 37
12.4 Further Reading . . . 38

13 Closing Perspective 39

Verified against primary sources for all sections; unverified claims are flagged inline.

1 The Short Version

If you have used Claude Code, Pi feels sparse on first contact: no built-in plan mode, sub-agent workflow, per-tool permission prompts, or general web search. It does include a Model Context Protocol (MCP) client for external tool servers, and optional codemode for composing tool calls. Pi is a programmable harness (the program around the model that runs the agent loop) that you extend. This primer connects the official reference documentation to practical decisions for someone who already knows Claude Code.

Pi is a terminal coding agent and an agent toolkit. Its default tools are `read`, `write`, `edit`,

and `bash`; optional built-in tools and MCP can expand that set. It provides persistent sessions, a terminal UI, and several ways to add your own behaviour: skills, prompt templates, TypeScript extensions, and packages. You can add a search skill, browser bridge, approval gate, subagent workflow, or local-model provider without forking the program.

Pi was created by Mario Zechner and is MIT-licensed. Since April 2026 the project has been owned by the company Earendil, and the repository lives in the `earendil-works` GitHub organisation. This edition was verified on 16 October 2026 against Pi 1.3.0, released on 15 October. Links under the current documentation track later changes; commands explicitly pinned to `v1.3.0`, an image digest, or a package version are reproducible examples rather than assertions about a later release. Check `pi --version` and the relevant release notes before applying an example to another version.

Read "How Pi works" for the control model, then "Installation and first run" for a supervised task before choosing add-ons. If you need automation, go next to "Running Pi non-interactively"; if you are migrating a Claude Code workflow, use the mapping table and "Customisation". The web-research and comparison sections are optional routes. "Sessions" and "Safety" apply to every route, and the Rocker project puts the decisions together in a reproducible example. Use "Reference" for configuration, terminology, and deeper sources.

1.1 What This Primer Is Not

This is not a benchmark, a list of every Pi package, or a guarantee that a model, provider, or community extension remains available. It is not a TypeScript API reference.

Pi is not safer than other agents merely because its core is small. It normally runs with the permissions of the launching user. Containment is an operating-system, container, or virtual-machine concern.

You need a terminal, Git, and comfort installing software. Familiarity with Claude Code (`CLAUDE.md`, skills, slash commands) is assumed. You do not need to know Pi or write TypeScript; the extension section assumes only that you can read a little of it when reviewing code.

2 How Pi Works

Before installing anything, it helps to know what Pi is, what it leaves out on purpose, and where a Claude Code habit will or will not carry over. The subsections below take those in turn.

2.1 Pi Is a Harness, Not a Model

An AI coding system has four separable parts:

1. **Model:** predicts text and tool calls. It may be hosted or local.
2. **Harness:** builds prompts, presents tools, sends requests, streams output, and records state.
3. **Tools:** give the model effects such as reading files, editing, running commands, searching, or browsing.
4. **Environment:** supplies permissions, network, filesystem, credentials, operating system, and repository.

Pi is mainly the second part, with a useful default tool set and a terminal interface. Its lower-level packages can also be embedded in other applications. Pi is not an inference server (a program that hosts a model and answers requests) and does not make an ordinary model browser-capable. If the model has no search capability and Pi has no search tool, the agent has no dependable web search. It can run `curl` through the `bash` tool, but that is an execution primitive, not a research system.

A request follows this shape:

```
your instruction
  -> session and system prompt
  -> provider and model
  -> model response or tool call
  -> Pi executes the tool
  -> result enters the session
  -> model continues
```

The model decides what it wants to do, Pi executes the request, and the operating system decides what the process may actually do.

2.2 What Pi Provides

Pi supplies a terminal UI, provider and model selection, context-file discovery, persistent sessions, and a small default tool set. Pi enables four tools by default: `read`, `bash`, `edit`, and `write`. The `grep`, `find`, and `ls` tools are built in but disabled unless you allow them with `--tools` or the `defaultTools` setting.

That is enough for:

```
inspect repository -> edit file -> run tests -> inspect failure -> edit again
```

Pi includes an MCP client as a built-in extension, while planning, subagents, browser automation, and general web search remain workflow choices. Optional `codemode` lets the model compose tool calls in JavaScript; it is off by default unless selected or enabled by an MCP server. See the CLI tool reference and codemode reference. You can install a package, write

an extension, create a skill, or keep other capabilities outside Pi and feed their results into the session.

Pi runs interactively, prints one-shot output, emits JSON events, communicates over remote procedure call (RPC), or is embedded through its TypeScript software development kit (SDK), so it is both a terminal program and a component of larger applications.

2.3 Why Pi Is Built This Way

The design is an argument, not an accident. The author's essay What I learned building an opinionated and minimal coding agent (November 2025) made it in five moves. Read these as the original design rationale, not a current feature list:

- **A tiny prompt.** The system prompt plus tool definitions come in below 1,000 tokens, and, the author argues, four tools suffice for coding work.
- **No MCP in the original core design.** The Model Context Protocol (MCP) is the standard for plugging such servers into an agent. The essay measured Playwright MCP at 21 tools and 13.7k tokens, and Chrome DevTools MCP at 26 tools and 18k tokens, "7-9% of your context window gone before you even start working". Pi 1.0 now ships a built-in MCP client with selective tool exposure; the essay remains useful as historical design rationale, not as a current feature list.
- **No built-in subagents, plan mode, or to-dos.** Subagents hide what the delegate did; plans and task lists belong in files you can read and edit.
- **No background bash.** The `bash` tool runs synchronously; use `tmux` for long-running processes.
- **YOLO by default.** Pi asks no permission before file operations or commands, and expects you to contain it externally.

The essay also reports a complete Terminal-Bench run as evidence. Treat that as the author's argument, not a neutral benchmark; the trade-off is that you supply much of the workflow yourself. The MCP position has since changed: current built-in MCP support uses selective tool exposure rather than requiring every server schema in every prompt. On licensing, the author states that Pi's core "will stay MIT licensed", while some future commercial features may be Fair Source or proprietary.

2.4 Coming From Claude Code

An existing Claude Code repository partly carries over: Pi loads `CLAUDE.md` as a context file and implements the Agent Skills format, but does not scan `.claude/skills`. In the table, *core* means Pi supplies the behaviour on installation, *extension/package* means you

must select and review code that adds it, and *absent* means Pi has no bundled equivalent. A row marked core does not imply identical semantics: commands with familiar names still need a small test before a migration.

Claude Code	Pi	Where it lives
`CLAUDE.md`	`AGENTS.md` or `CLAUDE.md`, both loaded from `~/.pi/agent`, the working directory, and its parents; `-nc` (`--no-context-files`) disables them	Core
Skills (`SKILL.md`)	Same Agent Skills format. Pi reads `~/.pi/agent/skills/`, `.pi/skills/`, `~/.agents/skills/`, and `.agents/skills/`, not `~/.claude/skills`	Core
Custom slash commands	Prompt templates in `prompts/`	Core
Hooks (`PreToolUse` and others)	Write or install a TypeScript extension using events such as `pi.on("tool_call", ...)`; there is no drop-in hooks file	Extension/package
Plugins	Pi packages distribute extensions, skills, prompts, and themes; `pi install` manages them	Core package mechanism; contents vary
`claude -p`	`pi -p` (`--print`)	Core
`--continue`, `--resume`, `/compact`	Resume or reduce session context; check the different branch and summary semantics below	Core
Rewind and branching (`/rewind`, `/branch`)	`/tree`, `/fork`, `/clone` branch the conversation; none of them rolls back file changes (the `git-checkpoint.ts` example does)	Core
`settings.json`	`~/.pi/agent/settings.json` and `.pi/settings.json`	Core
Permission prompts and modes	No bundled per-tool approval; `permission-gate.ts` shows an in-process gate, while containment needs an OS boundary	Absent in core; example extension
Plan mode	A plan file or the `plan-mode/` example	Absent in core; example extension

Claude Code	Pi	Where it lives
To-dos	A task file or the `todo.ts` example	Absent in core; example extension
Subagents	Separate Pi processes, the `subagent/` example, or a reviewed package such as `pi-subagents`	Absent in core; extension/package
MCP servers	Built-in stdio and Streamable HTTP client; `mcp.json`, `pi mcp`, and `/mcp`	Core through a built-in extension
Tool-call composition	Optional `codemode` JavaScript tool; `tool_search` is a distinct discovery tool	Core, optional built-in tools
`WebFetch` and `WebSearch`	Add a search skill, MCP server, or package; no general search backend is bundled	Absent in core

The migration order follows the table: first use existing context files and a read-only task; then copy one reviewed skill; finally replace only the hooks, MCP servers, or plugins you demonstrably need. Pi keeps credentials, trust decisions, and sessions under `~/.pi/agent/`, separate from `~/.claude`. A Claude Code permission rule in settings does not become a Pi permission boundary, and its session history does not move across automatically. Check Pi's loaded resources in the startup header and the effective tools before giving the migrated project write access.

3 Installation and First Run

Pi is a Node.js package. The npm route needs Node.js 22.19 or newer; the installer checks for Node and offers to install it. The quickstart distinguishes the managed installer, which pins dependencies, from npm, which no longer pins transitive dependencies. The two routes are alternatives, not a sequence:

```
# Option A: the installer (macOS and Linux). Download and read it before running.
curl -fsSL https://pi.dev/install.sh -o pi-install.sh
grep -n 'npm install\|curl' pi-install.sh   # ~1,700 lines; find what it runs
sh pi-install.sh

# Option B: npm (Node.js 22.19 or newer)
npm install -g --ignore-scripts @earendil-works/pi-coding-agent

# Either way:
```

```
pi --version
pi --help
```

Older tutorials use `@mariozechner/pi-coding-agent` or the `badlogic/pi-mono` repository. That npm package is deprecated and frozen at 0.73.1; new releases exist only as `@earendil-works/pi-coding-agent`, and the repository now lives at `earendil-works/pi`. Some companion repositories, including `badlogic/pi-skills`, still sit under the original account.

The commands above target macOS and Linux. Pi also documents native Windows and WSL, Android through Termux, and tmux key handling. Native Windows uses Git Bash for its `bash` tool unless you configure otherwise; WSL uses its Linux tools. Termux inherits Android storage permissions, and tmux may conflate modified Enter keys until extended keys are configured. Follow the platform page before assuming the example shell or shortcuts behave identically there.

`pi update` updates a managed Pi installation (`pi update --extensions` updates packages). In 1.0.1 and later, a global npm installation recommends migrating to the managed installer; follow your installation method's update instructions. Pi 1.0 also defaults to a fullscreen terminal UI; `pi --tui-mode regular` or `tuiMode: "regular"` preserves normal terminal scrollback. These changes are documented in the changelog. To remove an npm install, run `npm uninstall -g @earendil-works/pi-coding-agent`; that leaves `~/.pi/agent/` (credentials, sessions, settings) in place.

3.1 A Controlled First Session

The first session should prove the complete control loop, not demonstrate how much work Pi can do unattended. Use a disposable project:

```
mkdir -p ~/tmp/pi-first-run && cd ~/tmp/pi-first-run
git init
printf '# Pi test project\n' > README.md
pi --version
```

Then work through this sequence:

1. **Authenticate and choose one model.** Use `/login` or one environment variable (see "Hosted authentication" in the next section), then `/model`. Start with one tool-capable hosted model or one already-tested local model. Do not add routing or fallbacks before the basic loop works.
2. **Orient with restricted tools.** Start Pi with `pi --no-mcp --tools read,grep,find,ls`, then send the prompt below. This removes editing and shell tools and disables built-in MCP for the run; `--tools` alone now keeps MCP tools unless it explicitly filters them.

Loaded extensions still execute with your permissions, so this is a tool-selection check, not operating-system containment.

3. **Check what Pi loaded.** The working directory is the project: Pi discovers instructions and configuration from it and groups sessions by it. The startup header lists the context files and skills that loaded. If the folder holds `.pi/` resources or `.agents/skills`, Pi first asks whether to trust the project (see "Safety and containment").
4. **Make one supervised change.** Restart without `--tools`. Choose a small documentation or test change. Ask Pi to explain its plan, edit only the named files, run one check, and stop.
5. **Inspect independently.** Run `git status`, `git diff`, and the project's check command yourself. Keep or revert the change on evidence, not on Pi's confidence.

```
Inspect this repository. Report the files, likely project type,
and three useful next checks.
```

Keep the first change deliberately observable: for example, ask Pi to add one sentence to the disposable README and then stop. Before accepting it, check the diff yourself and compare the reported check command with what actually ran. If Pi tries to install a package, alter configuration, or inspect files outside the toy project, abort and inspect the session. The exercise establishes three separate facts: whether the selected model can use tools, whether Pi loaded the intended local context, and whether the files match the task contract. A fluent answer alone establishes none of them.

If the read-only orientation fails, isolate the layer before adding capabilities. A missing model is an authentication or catalogue problem; a tool-call error may be a model/protocol mismatch; an unexpected instruction may have come from an ancestor context file; and a missing project skill may be a trust decision. Run `pi --verbose` to inspect loaded resources, `pi auth check --provider <name>` without printing credentials, and the relevant one-tool check. A fresh `--no-session` run helps distinguish stale conversation context from a reproducible setup problem. Only then repeat the supervised edit. This sequence is intentionally small enough that you can explain every file and network path it used.

3.2 Working in the Terminal

You can send input while Pi works. `Enter` sends a steering message, which guides the next response once the current tool calls finish; `Alt+Enter` queues a follow-up that waits until Pi has finished all pending work; `Escape` aborts. `@` searches for a file to add to the prompt, `!git status` runs a command and sends its output to the model, `!!git status` runs it without sending the output, `/model` and `/thinking` change the model and its reasoning level, and `Shift+Enter` adds a line break.

Steering is useful when Pi has made a wrong assumption but is already working; the follow-up queue is for a separate instruction that should run only after this one finishes. Neither guarantees that an already-started command can be recalled. Check the transcript to see what ran before steering took effect. Terminal proxies and tmux can change modified-key behaviour, so use `/hotkeys` and the platform guidance if `Shift+Enter` or `Alt+Enter` submits unexpectedly.

4 Models and Authentication

Pi reaches a model through a provider. This section covers signing in to a hosted provider first, then compatible and local models.

4.1 Hosted Authentication

Either export an API key, or sign in from inside Pi. These are alternatives:

```
export ANTHROPIC_API_KEY='<your-api-key>'   # environment variable: suits ephemeral runs
pi
/login
```

`/login` stores an API key or, for some providers, a subscription credential in `~/.pi/agent/auth.json`. Protect that file, and keep secrets out of settings files, prompts, skills, and committed scripts. When several sources exist, Pi uses a `--api-key` flag first, then `auth.json`, then a `models.json apiKey`, then environment variables, so a stale stored login beats a freshly exported variable. `pi auth check --provider <name>` reports `ready`, `not_ready`, or `invalid` without printing a secret. Run `/model` afterwards to select a model.

Claude subscriptions need a measured check. Anthropic's 7 October 2026 policy statement says that Claude Agent SDK, `claude -p`, and third-party app usage can still draw from subscription limits. It also announces monthly API credits for Max and Team plans. Separately, Pi's Anthropic subscription warning cautions that third-party harness usage *may* use paid extra usage. Those sources establish the policy statement and the warning; they do not establish how one Pi OAuth session was metered. To check your own account, record the subscription and extra-usage balances, send a small, low-cost Pi prompt, then compare the account's usage display after it updates. Stop if the charge path is unclear. For budgeted API use, an API key gives you a separate billing path and a clearer cost record; it does not convert a subscription into API credit automatically.

Pi's built-in catalogue covers more than 15 providers, including Anthropic, OpenAI, Google Gemini, Azure OpenAI, Vertex, Bedrock, OpenRouter, Mistral, Groq, xAI, DeepSeek, and GitHub Copilot. It is bundled and refreshed from pi.dev (`pi update --models` forces a

refresh), so use `/model` or the model catalogue for current IDs and prices rather than copying an old model ID.

4.2 Hosted and Local Models

Provider flexibility is not the same as model interchangeability. Tool calling, streaming, context length, authentication, image support, and reasoning controls vary. A model that answers chat questions may still be a poor coding-agent model if it produces unreliable tool calls.

Choose the connection by what the endpoint actually speaks. Use `/login` and `/model` for a bundled provider. For an OpenAI-, Anthropic-, or Google-compatible endpoint, put the base URL and model in `models.json`; that is configuration of an existing protocol, not a new provider implementation. For a directory of GGUF files managed by llama.cpp's router, use `/login llama.cpp`, `/llama`, then `/model`. If you need a selectable name that routes each request among physical models, virtual models are the next layer. If the service needs different authentication, discovery, or a wire protocol Pi does not support, see custom providers. Neither advanced layer is needed just to point Pi at a compatible URL.

4.2.1 OpenAI-Compatible Local Servers

Ollama, LM Studio, vLLM, and SGLang (inference servers) can expose OpenAI-compatible endpoints. Pi reads them from `~/.pi/agent/models.json`, with no code:

```
{
  "providers": {
    "ollama": {
      "baseUrl": "http://127.0.0.1:11434/v1",
      "api": "openai-completions",
      "apiKey": "ollama",
      "models": [{ "id": "<installed-model-id>" }]
    }
  }
}
```

The `apiKey` is a dummy: Pi needs a key to treat the model as available, and Ollama ignores it. Replace the placeholder with the exact model ID served by your installation, then test its tool calling; a small model's ability to answer chat questions is insufficient. Check the context window the server allocates before blaming Pi for a lost instruction, because the server sets it. For an authenticated proxy, keep a real key out of committed JSON and use the environment interpolation documented in models. Test one harmless tool call and one long-context task before assigning production work; protocol compatibility alone does not prove equivalent streaming, tool schemas, or token accounting.

A useful arrangement is a local model for routine exploration, with a hosted model as an

explicit, per-task fallback for code you are willing to send to that provider. A local model does not make all data local: a search extension, browser, provider gateway, or external API can still receive content. Pi itself sends anonymous install and update telemetry by default (see "Safety and containment").

4.2.2 llama.cpp

Pi has a dedicated integration with llama.cpp's router server, which discovers several GGUF files (llama.cpp's model file format) and loads them on demand. Start `llama-server` without `--model`, `-m`, or `-hf`; passing a model starts single-model mode, and Pi's router integration then fails. It needs a llama.cpp build with router support.

```
llama-server \
  --models-dir ~/models \
  --no-models-autoload \
  --jinja \
  --host 127.0.0.1 \
  --port 8080 \
  -ngl 999 \
  -c 32768
```

`--models-dir` is where the GGUF files live, `--no-models-autoload` keeps loading explicit, `-ngl 999` offloads all layers to the GPU, and `-c 32768` sets the context per loaded model (omit it for the model's native context, which may need much more memory). The `--jinja` flag enables the model's chat template and tool calling; without it, tool calls are unreliable.

In Pi, run `/login llama.cpp` (default URL `http://127.0.0.1:8080`), then `/llama` to load a model, then `/model` to select it. `LLAMA_BASE_URL` and `LLAMA_API_KEY` configure the same connection without `/login`. Check the server with `curl http://127.0.0.1:8080/health` and `curl http://127.0.0.1:8080/models`.

Test the layers in order: health check, ordinary chat, streaming, Pi-compatible tool calls, and finally Pi executing a tool and feeding the result back. Keep one known-good hosted fallback while you stabilise the local path.

5 Running Pi Non-Interactively

The table compares Pi's main execution modes and the work each suits.

Way to run Pi	Interface	Use it when
Interactive	Terminal UI	A person is working with Pi
Print (`--print`, `-p`)	Final assistant text on stdout, then exit	A script needs the answer

Way to run Pi	Interface	Use it when
JSON (`--mode json`)	JSON Lines (one JSON event per line) on stdout, then exit	A program needs structured progress
RPC (`--mode rpc`)	JSON Lines commands on stdin, responses and events on stdout, long-lived	Another program drives Pi (RPC is remote procedure call: commands and replies over a pipe)
SDK	TypeScript library inside your Node.js or Bun process	You embed Pi in an application

```
pi --print 'Summarize this repository without editing it'
git diff --no-ext-diff | pi --print 'Review this diff. Return findings only.'
pi --no-session --print 'Explain this directory'     # in-memory session, nothing saved
pi --mode json 'Inspect the repository and report its test commands' > pi-events.jsonl
pi --mode rpc --no-session   # driven by another program; not interactive
```

When stdin or stdout is redirected and no mode is chosen, Pi uses print mode, and piped stdin is prepended to the prompt. For read-only review, restrict the tools:

```
pi --no-mcp --tools read,grep,find,ls --print 'Review without changing files'
```

`--tools` selects built-in, extension, and custom tools, but retains MCP unless an entry begins with `mcp__`. Use `--no-mcp` to disable the built-in MCP client for this review; a replacement extension needs its own controls. The CLI reference lists every flag. Tool selection still runs the agent process with your operating-system permissions.

The distinction between print and JSON matters for automation. Print yields the final assistant text and exits; an error or aborted final answer makes its exit status non-zero. JSON yields newline-delimited events for one invocation, including tool activity and the final message. A failed assistant response is represented in those events and does not, by itself, make the JSON process fail. If a script decides success from JSON mode, inspect the final `message_end` and `agent_settled` records as well as the process exit status. Neither mode accepts a new command after its run. See CLI integration and the event reference for the exact record shapes.

RPC keeps a Pi subprocess alive and accepts one JSON object per input line. Its `prompt` response reports acceptance, queuing, or handling, not task completion; `agent_end` can precede retry, recovery, or queued work. Wait for `agent_settled` before treating a run as finished. This small Python client illustrates the protocol for one prompt; it deliberately uses `--no-session` and a read-only tool set. Run it in a disposable directory with an authenticated model, and use the RPC command reference if you add state or session operations:

```python
import json
import subprocess

process = subprocess.Popen(
    ["pi", "--mode", "rpc", "--no-session", "--no-mcp",
     "--tools", "read,grep,find,ls"],
    cwd="/path/to/disposable-project",
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True,
)
try:
    command = {"id": "review-1", "type": "prompt",
               "message": "List the repository's test commands; change nothing."}
    process.stdin.write(json.dumps(command) + "\n")
    process.stdin.flush()
    accepted = False
    for line in process.stdout:
        event = json.loads(line)
        if event.get("type") == "response" and event.get("id") == "review-1":
            if not event.get("success"):
                raise RuntimeError(f"Pi rejected prompt: {event}")
            accepted = True
        elif event.get("type") == "agent_settled" and accepted:
            if event.get("aborted"):
                raise RuntimeError("Pi aborted the run")
            break
    else:
        raise RuntimeError("Pi closed stdout before agent_settled")
finally:
    process.terminate()
    process.wait()
```

The client must continue draining stdout, including unrelated session events, while the agent runs; a full pipe can stall the child. In a long-lived client, correlate replies by command ID, subscribe before sending the prompt, process extension UI requests if your extensions ask questions, impose a timeout, and inspect the final assistant message for provider or tool failure. Pi's checked typed RPC client example provides those lifecycle details for Node.js applications.

Use the TypeScript SDK when Pi should run in your Node.js or Bun process. It is not a CLI output mode. `prompt()` resolves after the run, and you dispose of the session when finished. Supply `cwd` explicitly when the target repository differs from the host process's current directory: that path determines resource discovery, context files, session grouping, and built-in tool paths. A persistent session is the default, so choose an in-memory manager when you do not want a transcript file:

```typescript
import { createAgentSession, SessionManager } from "@earendil-works/pi-coding-agent";

const { session } = await createAgentSession({
  cwd: "/path/to/disposable-project",
  sessionManager: SessionManager.inMemory(),
});
```

```
try {
  await session.prompt("What files are in the current directory?");
  console.log(session.getLastAssistantText());
} finally {
  session.dispose();
}
```

The SDK shares the host process's trust and credentials. In particular, embedding does not isolate an extension from your application, and the SDK does not load Pi's built-in MCP, codemode, or tool-search extensions automatically. Add them to the resource loader only when your application needs them; the SDK and MCP pages give the current setup. Choose RPC when a separate Pi process and a language-neutral protocol are useful; choose the SDK when you need direct TypeScript control of sessions, tools, and events.

6 Customisation: Use the Smallest Layer That Works

Pi's customisation layers run from plain instructions to executable code, and each step up adds power and review burden. The subsections below go through the layers in that order, then turn the choice into a table and a starter set.

Keep the mechanisms separate. A context file is always-on project guidance. A prompt template expands a repeated request. A skill supplies task-specific instructions and perhaps helper scripts. An extension runs TypeScript in Pi's process and can add tools or intercept events; a package distributes one or more of these resources. MCP connects an external tool server, while codemode is a separate Pi tool that writes JavaScript to find, call, and combine available tools. A subagent is another agent run or process coordinated by a workflow; none of the preceding mechanisms creates one automatically. Choose based on what must execute and who needs to review it, not on the label a Claude Code feature used.

6.1 Context Files and Prompt Templates

`AGENTS.md` and `CLAUDE.md` suit durable repository guidance: test commands, style, directory boundaries, and review requirements. Pi loads either from `~/.pi/agent`, the working directory, and its parents. `AGENTS.override.md` replaces them in the same directory only. `SYSTEM.md` replaces Pi's default system prompt and `APPEND_SYSTEM.md` extends it, at user or project level. Keep credentials and volatile provider details out of all of them.

Prompt templates play the role of Claude Code's custom slash commands. A Markdown file in `~/.pi/agent/prompts/` or `.pi/prompts/` becomes a command named after the file, and `$1`, `$@`, and `${1:-default}` substitute arguments. This one, saved as `review.md`, gives you `/review` and `/review concurrency`:

```
---
description: Review staged git changes
argument-hint: "[focus]"
---
Review the staged changes. Focus on ${1:-correctness, security, and error handling}.
```

Run `/reload` after editing a template in an active session.

6.2 Skills

A skill is on-demand instructions with optional scripts, references, and assets. Pi implements the open Agent Skills specification, the format Claude Code skills use, so a Claude Code skill directory generally works unchanged. Pi does not scan `~/.claude/skills`: copy the directory into `~/.pi/agent/skills/` or `~/.agents/skills/`, or point Pi at it with `--skill <path>` or the `skills` setting. Only each skill's name, description, and path sit in the prompt; the model reads the full `SKILL.md` when a task matches, or when you force it with `/skill:name`. Setting `disable-model-invocation: true` restricts a skill to that explicit command.

A project skill is a directory such as `.pi/skills/web-research/` holding a `SKILL.md`, plus optional `scripts/` and `references/`. The `SKILL.md` stays a short procedure:

```
---
name: web-research
description: Research a question from dated primary sources. Use for current docs and releases.
---

# Web research

Follow references/source-policy.md. Run scripts/search.sh "<query>".
```

If a skill needs a model-callable tool, it is becoming an extension. Project skills load only after you trust the project.

6.3 Extensions

Extensions are TypeScript modules executed inside Pi, which runs them directly with no build step. They can register tools, slash commands, lifecycle handlers, custom UI, provider adapters, and context or compaction behaviour. For a Claude Code user, one mechanism covers hooks, custom tools, and plugin code.

```
pi --extension ./my-extension.ts     # or -e; repeatable
~/.pi/agent/extensions/       # global
.pi/extensions/               # project-local; loads only after you trust the project
```

A subdirectory containing `index.ts` also loads, and `/reload` picks up changes. The `project_trust` event can be handled only by personal and command-line extensions.

A custom tool has a name, a parameter schema, and an `execute` function. This one counts the words in a string, adapted from Pi's `hello.ts` example:

```ts
import { Type } from "@earendil-works/pi-ai";
import { defineTool, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

const countWords = defineTool({
  name: "count_words",
  label: "Count words",
  description: "Count the words in a piece of text.",
  parameters: Type.Object({
    text: Type.String({ description: "The text to count" }),
  }),

  async execute(_toolCallId, params, _signal, _onUpdate, _ctx) {
    const n = params.text.split(/\s+/).filter(Boolean).length;
    return {
      content: [{ type: "text", text: `${n} words` }],
      details: { words: n },
    };
  },
});

export default function (pi: ExtensionAPI) {
  pi.registerTool(countWords);
}
```

Throwing from `execute` is how a tool reports failure; returning an object does not mark an error. A tool that touches the filesystem must validate its inputs, because it lets the model reach anything the user can.

The counterpart to Claude Code's permission prompts is a `tool_call` handler, which can change a call's input or block it. This gate, trimmed from the shipped `permission-gate.ts`, asks before a dangerous shell command and blocks it when there is no UI to ask (print and JSON modes; an RPC client can answer the dialog):

```ts
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function (pi: ExtensionAPI) {
  pi.on("tool_call", async (event, ctx) => {
    if (event.toolName !== "bash") return undefined;
    const command = event.input.command as string;
    if (/\brm\s+(-rf?|--recursive)|\bsudo\b/i.test(command)) {
      if (!ctx.hasUI) return { block: true, reason: "Dangerous command blocked" };
      const choice = await ctx.ui.select(`Allow this command?\n\n  ${command}`, ["Yes", "No"]);
      if (choice !== "Yes") return { block: true, reason: "Blocked by user" };
    }
    return undefined;
  });
}
```

Start from the shipped examples rather than from scratch: `permission-gate.ts`, `protected-paths.ts` (blocks writes to `.env`, `.git/`, and `node_modules/`), `confirm-destructive.ts`, `plan-mode/`, `subagent/`, `todo.ts`, `handoff.ts`, and `claude-rules.ts`. From a checkout of the repository, `pi -e packages/coding-agent/examples/extensions/permission-gate.ts` gives you the gate above. These are prompts inside the Pi process, not a security boundary: a regular expression is easy to evade, and the extension runs with your permissions. An

extension can read credentials, alter prompts, intercept tool calls, and start child processes. Review it like application code.

6.4 Packages

A package bundles extensions, skills, prompt templates, and themes (JSON colour files) for distribution through npm or Git; it is the counterpart of a Claude Code plugin. The package catalogue lists packages that carry the `pi-package` npm keyword. It is for discovery, not a security review.

```
pi install npm:@scope/package@1.2.3      # scoped npm package, version-pinned
pi install git:github.com/vendor/repo@<reviewed-commit>
pi install -l npm:@scope/package@1.2.3   # project scope; read only after trust
pi -e npm:@scope/package                  # try for one run, without installing
pi list
pi config                                 # enable or disable resources
pi remove npm:@scope/package@1.2.3
```

Versioned npm specifications, Git tags, and commits are pinned: `pi update --extensions` reconciles installations but does not move a configured ref. Read the source, installation scripts, and network calls before installing, and test third-party packages in a disposable project. A package declares its resources through conventional `extensions/`, `skills/`, `prompts/`, and `themes/` directories or a `pi` key in `package.json`.

6.5 MCP and Subagents

Pi's built-in MCP client connects to stdio or Streamable HTTP servers. A server provides operations; Pi's client merely connects and exposes them. Choose a reviewed server with the narrowest credentials and tool set for the task. The following is the verified Pi syntax, with an illustrative URL that you must replace with a real, read-only server endpoint:

```
pi mcp add docs --url https://mcp.example.com/docs
pi mcp list
pi
```

Inside Pi, `/mcp` shows connection state, tools, and exposure; after changing a server outside an active session, run `/reload`. `pi mcp add` writes a user server to `~/.pi/agent/mcp.json`; `-l` writes `.pi/mcp.json`, which loads only after project trust. `pi mcp list` actually connects and reports errors, so inspect its output before asking the model to use a tool. For an stdio server, the MCP guide uses `pi mcp add <name> -- <command> [args...]`. Do not put bearer tokens or OAuth tokens in a project file. The built-in OAuth flow stores its credentials in `~/.pi/agent/mcp-auth.json`; inspect the server's documented authentication route before connecting.

MCP exposure is a prompt and tool-selection decision. The default `codemode` exposure lets codemode scripts reach a server's tools without putting every schema in the model's

direct tool list. `direct` exposes selected tools immediately, `deferred` exposes them through the separate `tool_search` tool, and `hidden` makes them unavailable. Enable codemode for a session with the documented `pi --tools +codemode`; within a codemode script, `searchTools("documentation search")` discovers available tools and `describeTool()` shows the chosen declaration. `searchTools()` is a codemode function, not the `tool_search` tool. These controls limit what the model can invoke; they do not sandbox the server or the Pi process. Review server outputs as untrusted material.

The community `pi-mcp-adapter` is an alternative implementation, not a prerequisite; installing it can replace Pi's built-in `/mcp` for sessions. Subagents remain a workflow choice: Pi ships an example `subagent/` extension, while `pi-subagents` is a community package. For one independent research branch, a second named Pi process in a separate working tree or `tmux` pane is enough. Give it one question, read-only tools, a source limit, and a return format; have the main session inspect the returned URLs and conclusions before editing. Keep implementation file ownership disjoint when multiple agents work concurrently. A subagent's summary is a lead, not verification, and a nested agent inherits whatever credentials and filesystem permissions its process receives.

6.6 The Starter Suite: Install by Need

There is no universally correct add-on bundle. Pi's quickstart says to start with the least powerful mechanism that meets your need, and its security page notes that extensions run inside the Pi process with your permissions. Every executable extension adds code to review and another source of prompt injection (instructions hidden in files, web pages, or tool output that the model may obey) or data leakage.

Need	Start with	Move to a heavier mechanism when
Repository conventions	`AGENTS.md` (or your existing `CLAUDE.md`)	You repeat a procedure across repositories
Reusable procedure	A project or user skill	The procedure needs a model-callable tool or lifecycle hook
Repeated review prompt	A prompt template	The workflow needs state, events, or custom UI
Public web research	A reviewed search skill or narrow MCP service	You need another provider or richer extraction

Need	Start with	Move to a heavier mechanism when
JavaScript-heavy or authenticated pages	`browser-tools` from `pi-skills`	You need screenshots or frontend QA (see the caveats in "Add a browser only when needed")
New model or endpoint	A `models.json` entry, or `/login llama.cpp`	The service needs a custom protocol or authentication flow
Approval gate or new tool	A reviewed extension	You want to share it: publish a package

Instructions are easier to audit than executable customisation. A worked path: to have Pi review pull requests, start with a paragraph in `AGENTS.md`, then a `/review` prompt template, then a skill if the procedure grows scripts, and an extension only if you need a gate or a tool.

The search-skill installation and package example are in "Web research" so you can choose a data path before installing anything. Start with the built-in MCP client when you need MCP. Add a subagent framework, memory system, or planning overlay only after a bounded manual run shows which recurring step it would improve.

7 Web Research: Assemble the Missing Capability

Pi ships no general browser or web-search tool. A task that needs current documentation can use `curl` through `bash`, a search skill, a configured MCP search service, or an optional hosted service. Choose by the source you need and what you are willing to send out. The following path starts with static sources, then introduces discovery and browser automation only when the task calls for them.

7.1 Plain HTTP and Git Are Good for Static Sources

When you already know the authoritative repository or URL, retrieve that source directly. This avoids search-result summaries and makes the version visible in the command. The first and third commands below pin Pi's source at `v1.3.0`; the release API deliberately asks for the most recent release, so record its returned tag and date before citing it.

```
curl -fsSL -o extensions.md \
  https://raw.githubusercontent.com/earendil-works/pi/v1.3.0/packages/coding-agent/docs/extensions.md
curl -fsSL https://api.github.com/repos/earendil-works/pi/releases/latest
git clone --depth 1 --branch v1.3.0 https://github.com/earendil-works/pi.git ~/src/pi
```

This suits raw Markdown, JSON APIs, release metadata, and source trees. The `v1.3.0` paths are version-pinned source examples; use the linked latest docs for subsequent releases. A plain HTTP request does not search, render JavaScript, keep signed-in sessions, or extract readable text from a complex page. A successful status can still return only an application shell: the near-empty HTML page that a JavaScript app fills in later. Do not ask Pi to "search the web" and accept model-memory guesses; give it an explicit tool, source policy, evidence format, and stop condition.

7.2 Start With a Maintained Search Skill

The `pi-skills` repository is kept under Pi's original author's account and includes Brave web search and browser automation alongside unrelated integrations. Review the specific skill and its helper dependencies at the commit you choose. Install it outside Pi's automatically scanned skills directories, then load only that skill for the research run; this keeps unrelated skill descriptions out of the model's prompt. The commands below assume the checked-out commit contains the `brave-search` paths shown in its README:

```
git clone https://github.com/badlogic/pi-skills ~/src/pi-skills
git -C ~/src/pi-skills checkout <reviewed-commit>
(cd ~/src/pi-skills/brave-search && npm install --ignore-scripts)
pi --skill ~/src/pi-skills/brave-search
```

The skill needs a Brave Search API key. Follow the skill's own setup instructions for the account and current terms, then set `BRAVE_API_KEY` only in the shell that launches Pi. Queries go to Brave. To keep a reviewed skill available in every session, copy only its directory into `~/.pi/agent/skills/` or add its path in the `skills` setting. Review the version before refreshing either copy.

`/skill:name` forces a skill to load, which matters because a model can fail to load a relevant skill on its own; text after the command is appended as your request (the same pattern as Claude Code's `/name`). Use `Shift+Enter` for line breaks so the text is one message:

```
/skill:brave-search

Research the current Pi extension API. Search official documentation first,
then the repository, issues, discussions, and release notes. For each important
claim record URL, date, source type, and confidence. Do not install packages or
modify files. Return a source table before the synthesis.
```

These are the helper scripts the skill tells the model to run; run them yourself to test your key:

```
~/src/pi-skills/brave-search/search.js \
  'Pi extension registerTool' -n 10 --content --freshness pm

~/src/pi-skills/brave-search/content.js \
  https://raw.githubusercontent.com/earendil-works/pi/v1.3.0/packages/coding-agent/docs/extensions.md
```

`-n` sets the result count (default 5, maximum 20), `--content` fetches each page as Markdown, `--freshness` filters by `pd`, `pw`, `pm`, `py`, or a `YYYY-MM-DDtoYYYY-MM-DD` range, and `--country`

sets the region (default US). Prefer a raw Markdown URL to a GitHub `blob` page, whose HTML adds navigation noise. Keep the API key in the environment, not in the skill.

If a skill is insufficient, the community `pi-web-access` package combines search, fetching, and extraction. npm listed version `0.40.0` when checked on 16 October 2026, so `pi install npm:pi-web-access@0.40.0` is a reviewed-pin example, not a promise that it remains the most recent release. Check the current npm release and its changelog before changing the pin. Its keyless Exa route sends queries to a hosted service; other provider modes, extraction, answer generation, and local caching have different data paths. Inspect `web-search.json`, provider selection, and any browser-cookie option before using it with private material. A package runs extension code inside Pi; installing it is a different trust decision from giving a skill instructions.

As of 16 October 2026, Earendil's Radius web-search documentation describes an optional first-party hosted route. Web search is off by default for an organisation, requires an owner or admin to enable it, and can be reached through Radius MCP or its separate Pi extension. Radius is an early-alpha service according to Pi's provider documentation; check its current terms, data retention, and account controls before using it. The built-in Pi MCP client alone does not supply Radius search.

7.3 Add a Browser Only When Needed

For JavaScript-heavy documentation, authenticated forums, or frontend verification, `browser-tools` drives Chrome through the Chrome DevTools Protocol on port 9222 and extracts content after page load. Four caveats:

- **Check platform support at the reviewed commit.** The browser tool repository and its launcher are the source for Chrome paths and flags. An earlier revision hard-coded a macOS path; verify the current launcher before relying on Linux or Windows. If you launch Chrome yourself, use a disposable profile and bind remote debugging only to a trusted local interface.
- **The default profile is persistent, not disposable.** Without flags it uses `~/.cache/browser-tools`; delete it between runs. `--profile` copies your real Chrome profile (cookies and logins) into it; do not use that flag with an agent.
- **Anything that can reach port 9222 can drive the browser.** The skill can dump cookies (`browser-cookies.js`) and run arbitrary JavaScript in a page (`browser-eval.js`) into the model's context. Treat the browser, the model, and every extension as one trust domain, and use a narrow account.
- **It uses bot-detection evasion** (`puppeteer-extra-plugin-stealth`), which matters

for site terms of service on authenticated forums.

Use this decision rule:

Need	Appropriate first tool
Static documentation, JSON, raw GitHub	`curl` or a narrow shell skill
Search-engine results	Search API skill
JavaScript-rendered public page	Browser automation, or a hosted readability converter
Authenticated forum	Disposable browser profile and explicit scope
Frontend testing	Browser automation with screenshots and logs

7.4 Search GitHub and Forums Directly

`gh` is the GitHub command-line client; run `gh auth login` first, because code search needs authentication.

```
gh search code registerTool --repo earendil-works/pi
gh search issues 'MCP' --repo earendil-works/pi
gh api repos/earendil-works/pi/discussions \
  --jq '.[] | "\(.number)\t\(.title)\t\(.html_url)"'
```

Search results are leads, not evidence: read the issue, check its date and status, and distinguish a maintainer statement from a user workaround. A useful research brief states the decision it must support, the source order (official docs and source, release notes, issues, then community material), the required freshness, the evidence format (URL, date, claim, confidence), and a stop condition such as ten sources or three independent confirmations. For example:

```
Determine whether Pi currently has a built-in MCP client.
Search the official repository and docs first, then issues and the package
catalogue. Do not infer core support from an extension. Return URL, date,
source type, exact claim, and confidence. Stop after the current README, docs
index, source tree, and five relevant issues or package entries.
```

The answer for Pi 1.3.0 is yes: the official MCP documentation describes the built-in client, configuration, transports, and tool exposure. Older material correctly described earlier releases without it. This brief guards against treating an old design essay or forum answer as current behaviour, and against confusing a community adapter with the shipped client.

8 Sessions, Hand-Off and Workflow

This section covers how Pi stores and branches conversations, how to carry work from one session to the next, and a five-step routine that fits sessions into a task.

8.1 Sessions and Compaction

Pi sessions are persistent conversation records stored as JSON Lines files under `~/.pi/agent/sessions/`, grouped by working directory. Override the location with `--session-dir`, `PI_CODING_AGENT_SESSION_DIR`, or the `sessionDir` setting. The active conversation is one branch of a tree in that file.

```
pi --continue        # reopen the latest session for this directory (as in Claude Code)
pi --resume          # pick a saved session
pi --name research   # name a session; /name does the same from inside
pi --no-session      # in-memory only
```

Command	Result	Use it when
`/tree`	Moves within the current session file	Related alternatives should stay together
`/fork`	New session from an earlier user message	The alternative should become separate work
`/clone`	Copies the active branch into a new session	You want a separate copy of the current state

The distinction matters when exploring an alternative fix. `/tree` keeps both paths in one session file and changes which path supplies the next prompt; it does not reset the repository. Choose an earlier user message to edit and resubmit it as a new branch, or select an assistant entry to continue after it. `/fork` makes a new session from an earlier user message, which helps when the alternative should have a separate name and hand-off. `/clone` copies the active branch to a new session at its present position, useful before a risky experiment. None of these undoes file edits: inspect Git separately and use a worktree when alternatives must have separate files. Pi may add a branch summary when you navigate; verify its retained decisions against the original branch if they matter.

`/name` gives a session a searchable label, while `/session` shows its file, ID, message count, token usage, and cost. `/resume` opens the picker, and `/import <path>` resumes a JSONL session supplied from elsewhere. Use `/export [path]` for an HTML or JSONL copy. Session names are navigation aids, not access controls; an exported JSONL file can carry the full conversation and should be handled like source and log data.

Pi compacts automatically as the active context approaches the model's limit: the default reserve is 16,384 tokens, with roughly the most recent 20,000 tokens kept without summarisation. These are configurable thresholds, not a promise about exactly which message survives. `/compact <instructions>` triggers a manual summary and lets you specify decisions, paths, or unresolved questions to preserve. Pi adds a compaction entry and retains original entries in

the session tree; the model's *next request* receives the summary and retained recent messages, not every original. This is why a long session can keep full history on disk yet still forget a detail in its working context. Put durable requirements in a file and check the summary after compaction rather than expecting the model to reconstruct it from the JSONL file.

8.2 Sharing and Hand-Off

Sessions may contain source code, prompts, command output, URLs, tool arguments, and accidental secrets. Treat them as sensitive data. Pi's sessions guide says `/share` uploads a session to a Radius artefact when Radius authentication is configured and otherwise uses a private GitHub gist. Pi 1.3.0 adds `sessionShareDestination` to force `gist` or `radius`, or retain `auto`; a private gist is still accessible to anyone with its URL, and Radius visibility depends on the organisation's artefact settings. Inspect the actual destination and scrub the transcript before sharing. `/bug` can include a transcript in a report, so review its options too. An `/export` file stays local until you move it, but it needs the same inspection. For a durable hand-off, write:

```
objective
files changed
commands run and results
open questions
assumptions
next safe step
```

The shipped `handoff.ts` example extension can create a new session with a generated prompt through `/handoff <next task>`. Review that prompt and its source links before using it; a summary is not a substitute for the original evidence. When handing a task to another person, a concise note often reveals less than a full session export.

8.3 A Practical Workflow

1. **Map the repository.** Use a read-only prompt to identify instructions, build commands, tests, generated files, and the smallest safe change.
2. **Research in a separate session.** Use a search skill or API. Require dated sources and separate facts from inferences.
3. **Hand off evidence.** Write a short reviewed research note with URLs, dates, excerpts, and unresolved uncertainty.
4. **Implement in a branch or worktree.** Give Pi one bounded change, explicit file ownership, and proof commands. If you delegate a separate research question to a subagent or another Pi process, state its source limit and expected return format, and do not give it the implementation files.
5. **Verify independently.** Run formatter, tests, static checks, and rendering outside the

model's narrative. Review the diff and working-tree state. For delegated work, open the cited sources yourself, reproduce at least one decisive check, and compare the result with the main task contract before adopting it.

Name each session (`--name` or `/name`) so hand-off notes can refer to it. This keeps research context from overwhelming implementation context and makes the result auditable, if not reproducible.

9 Safety and Containment

Pi is not a sandbox and has no built-in permission boundary for filesystem, process, network, or credential access. It runs with the launching user's permissions. Bash, extensions, package installers, language servers, and child processes can affect whatever that user can affect. Claude Code, by contrast, offers per-tool permission prompts and an optional OS-level sandbox for shell commands (Seatbelt on macOS, bubblewrap on Linux); Pi has neither built in, and its security documentation says to use one of the isolation patterns below.

Three controls are easy to confuse:

- Project trust controls whether project-local resources load.
- Tool selection and MCP exposure control which tools are callable; `--tools` alone does not remove MCP tools.
- The operating system, container, VM, or sandbox controls what the process can touch.

9.1 Project Trust

Trust is a resource-loading decision: it stops a folder silently loading executable extensions, project packages, or project MCP servers. Pi asks for a decision when the working directory holds any of these: `.pi/settings.json` or `.pi/mcp.json`; `.pi/extensions`, `.pi/skills`, `.pi/prompts`, or `.pi/themes`; `.pi/SYSTEM.md` or `.pi/APPEND_SYSTEM.md`; or `.agents/skills` in the working directory or an ancestor. A bare `.pi` directory needs no decision. Declining skips those protected resources. The security guide describes the complete loading boundary.

Context files are not gated. `AGENTS.md`, `CLAUDE.md`, and `AGENTS.override.md` load regardless of trust unless you pass `-nc`, so a hostile repository can still steer the model through them even if you decline trust. One documented gap: the project `sessionDir` setting is read before trust is resolved.

You decide interactively, save a decision with `/trust` (stored in `~/.pi/agent/trust.json`),

or override for one process with `--approve` (`-a`) or `--no-approve` (`-na`). The `defaultProjectTrust` setting is `ask` (the default), `always`, or `never`. Print, JSON, and RPC modes cannot prompt, so under `ask` they skip protected resources unless a saved decision or `--approve` applies: a CI script that expects a `.pi/` extension to load silently will not load it. Trust is not containment: after startup, enabled tools still use the process's operating-system permissions. User-level extensions and trusted project packages also run inside that process; an extension can inspect prompts, tool calls, session history, files, and credentials. Treat package installation and project approval as code review decisions, and put the process inside an actual isolation boundary when access must be limited.

9.2 Isolation Options

For untrusted repositories, unattended runs, production credentials, or browser profiles, use a container or VM. A container is only as strong as its mounts, environment variables, network access, and user identity. Pi's containerisation guide documents four methods:

Method	Where Pi runs	What is isolated	Credentials
Plain Docker	Container	Pi, its tools, `!` commands, extensions	Passed into the container
Docker Sandboxes (`sbx`)	Managed sandbox	The same	Stay on the host; a proxy substitutes them
NVIDIA OpenShell	Local or remote sandbox	The same	Policy-controlled
Gondolin extension	Host	Built-in tools and `!` commands only, in a local micro-VM	Commands inherit host environment variables, so it is not a credential boundary

Plain Docker is the simplest. Pi's documentation builds the image from this `Dockerfile.pi`:

```
FROM node:24-bookworm-slim
RUN apt-get update \
  && apt-get install -y --no-install-recommends bash ca-certificates git ripgrep \
  && rm -rf /var/lib/apt/lists/*
RUN npm install -g --ignore-scripts @earendil-works/pi-coding-agent
WORKDIR /workspace
ENTRYPOINT ["pi"]
```

```
docker build -t pi-sandbox -f Dockerfile.pi .
docker run --rm -it \
  -e ANTHROPIC_API_KEY \
  -v "$PWD:/workspace" \
  -v pi-agent-home:/root/.pi/agent \
  pi-sandbox
```

`podman run` works the same way; on an SELinux host add `:Z` to the bind mount. Passing `-e ANTHROPIC_API_KEY` puts the key inside the container, where every process, extension, and tool call can read it. The named `pi-agent-home` volume keeps settings, credentials, and sessions between runs; mounting the host's `~/.pi/agent` instead would expose your real credentials, extensions, and sessions. Do not mount the whole host home directory or a real browser profile, and give research extensions only the keys and network routes they require.

9.3 Prompt Injection, Telemetry, and Reporting

Prompt injection can come from a README, issue, web page, command output, or model response; treat external text as untrusted input. Pi's security policy treats prompt injection, the lack of a built-in sandbox, and user-installed extensions or skills as generally outside its security boundary; report genuine bypasses through `SECURITY.md`, not a public issue.

Pi also makes outbound requests of its own. The settings reference says `enableInstallTelemetry` defaults to `true` for anonymous install/update reporting and selected provider attribution headers; `PI_TELEMETRY=0` overrides those. It does *not* disable update checks. `PI_SKIP_VERSION_CHECK=1` disables the pi.dev latest-version request, while `pi --offline` or `PI_OFFLINE=1` disables automatic network activity, including model-catalogue refreshes; a cached catalogue remains available. These controls are distinct from the network used by a chosen model provider, MCP server, web skill, or package. Do not treat `--offline` as a firewall for tools or extensions; use OS or network policy for that. `/share` and `/bug` are explicit upload actions whose transcript contents need inspection.

10 Pi Compared With Other Coding Agents

The useful comparison is not a feature checklist. Ask where the product puts workflow decisions and what the default execution boundary is.

Agent	Centre of gravity	Provider posture	Web posture	Customisation and controls
Pi	Programmable terminal harness and embeddable runtime (MIT)	Broad providers plus compatible and local endpoints	Add a skill, MCP service, extension, package, or shell workflow	TypeScript extensions, built-in MCP and optional codemode; no per-tool approval system
OpenCode	Ready-made terminal coding application	Broad provider and local support	Built-in `webfetch`; `websearch` only with the OpenCode provider or an opt-in Exa or Parallel flag	Per-tool permissions, plugins, skills, MCP
Aider	Git-aware pair programmer	Many providers and local endpoints	`/web <url>` scrapes a page into the chat; no built-in search	Configuration and scripts; narrower runtime
Claude Code	Opinionated terminal agent around Claude	Strong Anthropic alignment and supported cloud deployments	Built-in `WebFetch` and `WebSearch`, permission-gated	Permission modes, optional OS-level sandbox, subagents, hooks, skills, plugins, MCP
Codex	Product-connected coding agent from OpenAI	OpenAI-first; custom providers and local `--oss` models supported	Built-in web search, cached results by default	Instructions, skills, plugins, MCP; sandbox modes and approval policies built in

10.1 Pi Versus OpenCode and Aider

OpenCode feels like a complete coding application; Pi starts smaller and asks you to assemble the workflow. Choose OpenCode for a richer day-one experience. Choose Pi when the agent

loop itself must be programmable, when you want a narrow, provider-neutral harness, or when you are building an embedded agent. Aider's centre is the repository, selected editable files, a repository map, and a reviewable Git edit loop. Pi can reproduce that loop but can also become a research tool, browser bridge, or application component; choose Aider for focused supervised edits.

10.2 Pi Versus Claude Code and Codex

Claude Code and Codex supply the surrounding workflow themselves: permission prompts, sandbox options, web search, and established instruction systems. The mapping table in "Coming from Claude Code" shows where Pi leaves each of those to you: you select the model, assemble research access, review extensions, and define containment. That flexibility is not automatically cheaper or safer. Local models add hardware and quality costs; extensions can add exactly the capability you want while gaining full process access. The sibling primers cover the alternatives to Claude Code and Codex for Claude Code users in more depth.

10.3 Forks and Derivatives

Because Pi is MIT-licensed and embeddable, others build on it. `oh-my-pi` (Stencil Labs, MIT) is, by its own README, a fork that adds subagents, integrated development environment (IDE) and Language Server Protocol (LSP) support, and browser tooling. OpenClaw (see the OpenClaw primer) was described in January 2026 as having Pi "under the hood" (Armin Ronacher); its current documentation says only `@earendil-works/pi-tui`, the terminal-UI toolkit, remains a dependency. A claim that a project "runs on Pi" needs a date.

A fork can be useful when its integrated defaults match the task, but it changes the code and update path you are trusting. Compare the fork's actual release and security model with the Pi version in this primer before carrying over commands. Likewise, an application that once embedded Pi need not still share Pi's sessions, tools, or provider handling. Treat those relationships as versioned implementation facts rather than product categories.

11 Worked Example: A Reproducible Data-Science Project With Pi and Rocker

This project makes the boundary between an agent and an analysis runtime concrete. Pi runs on the host and helps inspect the repository, research package APIs, write code, and interpret failures. Rocker runs the R and Python analysis in a reproducible container, and Podman supplies the rootless boundary. Pi itself remains a host process with your permissions (see

"Safety and containment"); the container bounds only the analysis code. Because Pi runs `just qa` on the host, it also has host access to Podman and everything else you have.

Terms: **Rocker** publishes versioned R container images (`rocker/r-ver`, `rocker/tidyverse`); the tidyverse image bundles R, `readr` and `dplyr` (data reading and manipulation), and `testthat` (R's unit-test framework), and is large (about 3.8 GB), where `rocker/r-ver` is much smaller. **Podman** is a daemonless container engine with a Docker-compatible command line; **rootless** means it runs without root privileges on the host. A **digest** is a content hash (`sha256:...`) identifying an exact image, where a tag is a movable label. `renv` is R's dependency lockfile tool.

11.1 Project Goal and Contract

Build a small "daily temperature quality" project. The input is a fictional CSV of station readings; two independent implementations, one in R and one in Python, must produce the same summary. Do not call Python from R (for example through the `reticulate` package): independence is what makes their agreement meaningful. You write this contract before Pi writes any code:

- Dates are ISO `YYYY-MM-DD`; a malformed date is an error. The allowed `quality_flag` values are `ok` and `missing`; any other value is an error, as is an `ok` row with an empty `temp_c`. An error means a non-zero exit status and a message naming the problem.
- Rows marked `missing` are discarded. A station-day with only `missing` rows produces no output row. Group the rest by `date` and `station_id`.
- The output header is exactly `date,station_id,n_valid,mean_temp_c,min_temp_c,max_temp_c`, sorted by `date`, then `station_id`, byte-wise (C-locale order; R's default string ordering depends on the locale). Format temperatures with two decimals; use UTF-8, LF line endings, and no quoting.
- Tolerance: keys, `n_valid`, `min_temp_c`, and `max_temp_c` must match exactly; `mean_temp_c` may differ by at most `0.01`, because R and Python can round a tie differently.

The repository layout:

```
pi-temperature-starter/
  AGENTS.md
  data/raw/station_readings.csv
  data/expected/daily_summary.csv
  r/clean.R
  r/testthat/test-clean.R
  python/clean.py
  python/tests/test_clean.py
  scripts/compare_outputs.py
  outputs/              # generated; git-ignored apart from .gitkeep
```

```
renv.lock
requirements.txt
Containerfile
.containerignore
.gitignore
Justfile
README.md
```

`AGENTS.md` carries the rules Pi must follow, and Pi loads it automatically at startup:

```
# Project rules
- Run all analysis inside the container with `just qa`. Never run R or pip on the host.
- Do not edit `data/raw/`, `data/expected/`, `Justfile`, or `Containerfile`.
- Read the contract in `README.md`. Stop and ask if it is ambiguous.
```

Start with a deliberately small fixture. One group has two valid readings, so the mean, minimum, and maximum differ; the last row forms a group of only missing data:

```
date,station_id,temp_c,quality_flag
2026-01-01,A,5.1,ok
2026-01-01,A,6.3,ok
2026-01-01,A,,missing
2026-01-01,B,3.4,ok
2026-01-02,A,,missing
```

The expected output is part of the contract, not something Pi may invent after seeing its own result. Write it by hand:

```
date,station_id,n_valid,mean_temp_c,min_temp_c,max_temp_c
2026-01-01,A,2,5.70,5.10,6.30
2026-01-01,B,1,3.40,3.40,3.40
```

The fixture has no unknown flag or malformed date, so those failure cases live in the unit tests as small in-memory inputs. The acceptance gate is `scripts/compare_outputs.py`, which you write, not Pi. It reads `outputs/daily_summary_r.csv` and `outputs/daily_summary_py.csv`, checks each header, compares each row with the expected file (exact keys and counts, `mean_temp_c` within the tolerance), and exits non-zero on any difference. Row-by-row comparison against a sorted file also checks the sort order.

11.2 Give Pi Bounded Tasks

Use a separate, named session for each phase. The fixture, expected output, and comparison script are committed before the implementation task starts.

Phase 1 restricts tools: `pi --name contract --no-mcp --tools read,grep,find,ls`.

```
Inspect the repository rules and propose the file contract for this project.
Do not write files or install packages. Identify assumptions and test cases.
```

Phase 2 needs a search skill and `bash`: `pi --name research --skill ~/src/pi-skills/brave-search` (see "Web research"). Without a search skill, drop this step and supply the documentation URLs yourself; otherwise Pi answers from model memory.

```
Research the current official documentation for readr, dplyr, testthat,
Python csv handling, and pytest. Return URLs and version constraints only.
Do not modify the repository.
```

Phase 3 is implementation in a fresh session, `pi --name implement`, with the research hand-off note pasted in (see "Sharing and hand-off"):

```
Implement the R and Python cleaners from the checked-in contract. Add tests
for missing values, unknown flags, malformed dates, and cross-language
agreement. Use `just qa` (defined below) and stop if the contract is ambiguous.
```

The model drafts the code. The fixture, expected output, container build, and comparison decide whether it is correct. Pi also writes some tests, so review them: a test written to match the model's own output proves nothing.

11.3 Build a Digest-Pinned Rocker Image

Rocker's versioned tags are not immutable: the images are rebuilt periodically, so `4.6.1` today need not be byte-identical to `4.6.1` last month (Rocker's reproducibility guide says to use a digest if you need the same image). Pin the multi-architecture image index digest, so the pin works on amd64 and arm64. The digest below was recorded for the worked example on 24 September 2026; later tag movement does not change that pinned image. To update the example deliberately, inspect the current index digest and change *both* the Containerfile and the lockfile-generation command together. `podman image inspect` reports a single-platform manifest digest, which is the wrong source for the cross-platform pin.

```
skopeo inspect --format '{{.Digest}}' docker://ghcr.io/rocker-org/tidyverse:4.6.1
```

```
ARG ROCKER_IMAGE=ghcr.io/rocker-org/tidyverse:4.6.1
ARG ROCKER_DIGEST=sha256:94a4cdb7be4d1214f9039bca4937e8919253d2ad90d8bed084e9a8af685a50c7
FROM ${ROCKER_IMAGE}@${ROCKER_DIGEST}

USER root
# python3 is already in the image; the venv module is not.
RUN apt-get update \
 && apt-get install -y --no-install-recommends python3-venv \
 && rm -rf /var/lib/apt/lists/* \
 && python3 -m venv /opt/venv

ENV PATH="/opt/venv/bin:${PATH}"
WORKDIR /work

COPY requirements.txt .
# --require-hashes means every requirement must be pinned with a hash.
RUN pip install --no-cache-dir --require-hashes -r requirements.txt

COPY renv.lock .
RUN R -q -e 'install.packages("renv")' \
 && R -q -e 'renv::restore(prompt = FALSE)' \
 && rm -rf /tmp/downloaded_packages

USER rstudio
```

The virtual environment isolates the pinned Python packages from the Ubuntu system Python. Tag pinning alone does not make the R packages reproducible, because Rocker installs the selected R version from a moving CRAN snapshot; that is why the image uses `renv`. Four details matter:

- `renv::restore()` in a project that is not activated installs into the image's system library and can overwrite Rocker's own packages. An incomplete lockfile, or one taken from another machine, can downgrade packages that the tidyverse relies on, and the image then fails at load time. Generate `renv.lock` from a container running this same image, so it records Rocker's package set as well as yours:

```
podman run --rm -v "$PWD:/w:Z" -w /w \
  ghcr.io/rocker-org/tidyverse:4.6.1@sha256:94a4cdb7be4d1214f9039bca4937e8919253d2ad90d8bed084e9a8af685a50c7 \
  R -q -e 'install.packages("renv"); renv::snapshot(type = "all", lockfile = "renv.lock",
  prompt = FALSE)'
```

- The bootstrap installs whatever `renv` version the mirror serves that day; the lockfile records `renv`'s version but this step does not honour it.
- `requirements.txt` must carry hashes. Write direct dependencies (here `pytest`) in `requirements.in` and run `pip-compile --generate-hashes requirements.in`, or `uv pip compile --generate-hashes requirements.in -o requirements.txt`.
- Under `--userns=keep-id` (below) the process runs as your host user whatever `USER` says; `USER rstudio` (UID 1000 in this image) serves runtimes without keep-id. Use `.containerignore` as an allow-list (`*`, then `!requirements.txt` and `!renv.lock`), because the build context includes everything not ignored.

11.4 Run It With Rootless Podman

A small `Justfile` exposes the workflow without hiding the container boundary. These recipes were built and run against a toy implementation with Podman 5.7.0 on Ubuntu, and the toy project passed `just qa`:

```
set shell := ["bash", "-ceu"]

IMAGE   := "localhost/pi-temperature:dev"
ROOT    := justfile_directory()
# Podman already mounts a tmpfs on /tmp for read-only containers; --tmpfs adds a size cap.
SECURE  := "--rm --userns=keep-id --cap-drop=all --security-opt=no-new-privileges " + \
           "--read-only --network=none --tmpfs /tmp:rw,size=256m"
ENV     := "-e PYTHONDONTWRITEBYTECODE=1 -w /work"
# Read-only inputs: only the directories the container needs, not the repository root.
INPUTS  := "-v " + ROOT + "/r:/work/r:ro,Z -v " + ROOT + "/python:/work/python:ro,Z " + \
           "-v " + ROOT + "/scripts:/work/scripts:ro,Z -v " + ROOT + "/data:/work/data:ro,Z"
OUTPUTS := "-v " + ROOT + "/outputs:/work/outputs:Z"
RUN     := "podman run " + SECURE + " " + ENV + " " + INPUTS

build:
  podman build -t {{IMAGE}} -f Containerfile .

test-r:
  {{RUN}} {{IMAGE}} Rscript -e 'testthat::test_dir("r/testthat")'

test-python:
  {{RUN}} {{IMAGE}} pytest -q -p no:cacheprovider python/tests

pipeline:
```

```
  mkdir -p outputs
  {{RUN}} {{OUTPUTS}} {{IMAGE}} Rscript r/clean.R
  {{RUN}} {{OUTPUTS}} {{IMAGE}} python python/clean.py
  {{RUN}} {{OUTPUTS}} {{IMAGE}} python scripts/compare_outputs.py

qa: test-r test-python pipeline
```

Only `outputs/` is writable, so a buggy or malicious script cannot alter the fixture or expected output. Mounting only the needed sub-directories keeps `.git/config` (which may hold remote URLs or tokens) and `.git/hooks` out of the container, and matters because `:Z` relabels the mounted tree for SELinux hosts (and is ignored elsewhere). `outputs/` must exist before Podman starts, since Podman refuses a bind mount with a missing source, so `pipeline` creates it. `-p no:cacheprovider` and `PYTHONDONTWRITEBYTECODE` stop pytest and Python writing into read-only mounts, and `--network=none` means the runs cannot fetch anything.

The recipes call `podman` directly because `--userns=keep-id` is Podman-only. On Docker, use `--user "$(id -u):$(id -g)"` instead and drop `:Z` unless the host uses SELinux. Never mount the repository parent, home directory, SSH agent, cloud credentials, or a container socket.

11.5 Verify the Result Independently

Pi should run `just qa` and report its output, but its report of its own run is not evidence. Run `just qa` yourself and check:

1. Both test suites pass, and unknown flags and malformed dates fail as expected.
2. The `missing` rows do not contribute to `n_valid` or the statistics, and the `2026-01-02` station-day produces no row.
3. The R and Python outputs match the expected file within the declared tolerance (`compare_outputs.py` checks this).
4. `git diff --stat` shows only the intended source, lock, and test changes: the `Justfile`, `Containerfile`, `.containerignore`, `AGENTS.md`, and the fixture and expected files are unchanged, and nothing was written into the source tree.

Item 4 matters most. Pi runs on the host with no approval prompts, so it can edit the `Justfile` or `Containerfile` to loosen `--network=none`, `--read-only`, or the mounts. The container flags are a convention Pi follows, not a boundary it is subject to. Commit those files first and review any change to them as a separate diff.

If the project later uses real data, keep it outside Git and mount it read-only. Do not give Pi access to sensitive datasets or credentials merely because the analysis container can read them. This example teaches reproducibility and review; it does not make Pi a trusted data-

processing authority.

12 Reference

Look-up material for when something breaks, when you need to find a file, and when you want the next document to read.

12.1 Troubleshooting Order

Work from the process outwards: each step only makes sense if the previous one is green.

1. Executable: `command -v pi`, `pi --version`, `pi --help`.
2. Working directory and discovered context files: the startup header, `--verbose`, and `-nc` (`--no-context-files`) to exclude `AGENTS.md` and `CLAUDE.md`. If a project skill or extension is missing, check project trust (`-a` or `-na` forces a decision for one run).
3. Provider authentication, without printing secrets: `pi auth check --provider <name>`. Avoid `pi auth print-api-key` and `print-bearer-token`, which write secrets to stdout.
4. Model response and tool-call support.
5. Provider or search network path. `--offline` excludes catalogue refresh.
6. Underlying `curl`, `gh`, browser helper, or local server.
7. Extension or skill disabled and retested: `--no-extensions` (`-ne`), `--no-skills` (`-ns`), `--no-prompt-templates` (`-np`), or `-nt` for no tools; `pi config` toggles package resources.
8. Fresh or `--no-session` run to exclude stale context.
9. Independent reproduction of the repository failure.

Do not install another extension until you know which layer failed.

12.2 Configuration Map

The configuration reference distinguishes personal files in the agent directory from project resources under `.pi/`. The separate MCP guide documents `mcp-auth.json`, which holds OAuth credentials for remote MCP servers. Back up or migrate credentials and sessions intentionally; do not copy these files into a repository merely to reproduce a tool setup.

```
~/.pi/agent/                    agent directory (override: PI_CODING_AGENT_DIR)
  settings.json                 user settings, defaults, package declarations
  models.json                   custom providers and models
  auth.json                     saved API keys and OAuth credentials (protect it)
  mcp.json                      user MCP server definitions
  mcp-auth.json                 MCP OAuth credentials (protect it)
  trust.json                    saved project-trust decisions
```

```
  sessions/                     session files, grouped by working directory
  extensions/  skills/  prompts/  themes/
  AGENTS.md | CLAUDE.md         instructions applied in every directory
  SYSTEM.md, APPEND_SYSTEM.md   replace or extend the system prompt
~/.agents/skills/               portable Agent Skills location (also read by Pi)

.pi/                            project settings and resources (load after project trust)
  settings.json  mcp.json  extensions/  skills/  prompts/  themes/
  SYSTEM.md  APPEND_SYSTEM.md
.agents/skills/                 portable project skills (also trust-gated)
AGENTS.md | CLAUDE.md           project instructions (loaded regardless of trust)
```

12.3 Glossary

These terms recur across the practical route. The links point to the first fuller explanation in this primer; the external documentation remains the authority for exact commands and settings.

Term	Meaning and first explanation
Harness	The loop around a model that builds context and executes tool calls; Pi's architecture.
Tool	An operation the model can request through the harness; default tools.
Context file	Project or user instructions loaded into prompts; context files.
Prompt template	A reusable slash-command prompt with arguments; templates.
Skill	Task-specific instructions and supporting resources; skills.
Extension	Executable TypeScript that can add tools or hooks; extensions.
Package	A distributable bundle of Pi resources; packages.
MCP	Model Context Protocol, used to connect external tools; MCP setup.
Codemode	A Pi tool for JavaScript orchestration of other tools; codemode and discovery.
Subagent	A separate agent run assigned a bounded part of a task; subagent pattern.
RPC	A command-and-event protocol over a long-lived subprocess; RPC example.

Term	Meaning and first explanation
SDK	Pi's in-process TypeScript integration; embedding example.
Session branch	One path through a saved conversation tree; sessions.
Compaction	Summary that reduces history sent to the model while retaining original entries; compaction.
Project trust	A choice about loading project resources, not process isolation; trust.
Digest	A content-addressed image identifier, unlike a moving tag; Rocker example.

12.4 Further Reading

Pi documentation

- Pi repository, documentation index, quickstart, and how Pi works: start here.
- Pi changelog and 1.3.0 release: the 16 October 2026 verification baseline.
- Built-in MCP, codemode, and virtual models: server configuration, tool-call composition, and per-request routing.
- Security and isolated environments: project trust, and Docker, Docker Sandboxes, OpenShell, and Gondolin.
- Command line, configuration, sessions, and slash commands: flags, the agent directory, branching, and compaction.
- Models, providers, and llama.cpp: authentication, `models.json`, and the router integration.
- Skills, prompt templates, extensions, packages, and themes.
- CLI integration, JSON events, RPC, and the SDK: print, event streams, long-lived subprocess control, and embedding.
- Package catalogue and model catalogue; extension examples; releases.

Design and ecosystem

- Mario Zechner, What I learned building an opinionated and minimal coding agent: the design rationale behind everything in "Why Pi is built this way".
- Mario Zechner, I've sold out: the move to Earendil and the licensing plan.
- `pi-skills`, `pi-web-access`, `pi-mcp-adapter`, and the `oh-my-pi` fork.

- Agent Skills specification, the format shared with Claude Code.
- Radius, Earendil's hosted gateway for Pi.

Containers and reproducibility

- Podman run and Podman build references.
- Rocker: versioned images, reproducibility, rootless Podman, and extending images.
- `renv` and Docker, pip hash-checking, and the `just` manual.

Other agents and related primers

- Aider, OpenCode, Claude Code, and Codex CLI.
- Claude Code alternatives primer, Codex for Claude Code users primer, and OpenClaw primer.

13 Closing Perspective

Pi's distinctive feature is its programmable loop and selective tool exposure. Built-in MCP and optional codemode now supply tool integration and composition; you still decide which browser, search service, approval policy, planning workflow, and subagent strategy belong around it.

Pi suits you if you want a loop you can program: a provider-neutral harness, an embeddable runtime, a research workflow you assemble yourself. It suits you less if you want bundled per-tool approvals, subagents, and web search on day one. MCP is now supplied, but its servers still need configuration and review. Pi runs with your permissions, so put a container or VM around anything untrusted or unattended, and review extensions, packages, and MCP servers as code you are about to execute.

A sensible first week: install Pi and check `pi --version`; run the read-only orientation from "A controlled first session" on a repository you know; add one skill; then repeat the Rocker example with your own fixture. Read the author's design essay before deciding what else to add.